

UC: Representação e Processamento de Conhecimento na Web

20 de Maio de 2026, 9h, Ed. 2: sala 0.14

Mestrado em Engenharia Informática (1º ano)

Sinopsis

O objectivo principal deste teste é avaliar os conhecimentos obtidos durante as aulas na especificação e desenvolvimento de ontologias e de aplicações que tiram partido destas.

Antes de começares, lê atentamente até ao fim para ficares com uma percepção do todo que se pretende. Vais ver que tomarás decisões mais acertadas depois de uma leitura completa.

Os resultados finais deverão ser enviados ao docente da seguinte forma:

- Enviar por email para: jcr@di.uminho.pt;
- Colocar no subject/assunto: **RPCW2026::Normal::Axxxxx**, em que **Axxxxx** corresponde ao número do aluno (nalguns casos **PGxxxxx**);
- Enviar ao docente um link do github para um repositório novo criado especificamente para o exame com o seguinte conteúdo (esta preparação poderá valer 1 valor do exame): o repositório no GitHub deverá chamar-se **RPCW2026-Normal**;
- Dentro do repositório deverá haver um ficheiro, **README.md**, contendo uma descrição do que fez e como fez, contendo instruções se necessário, para o docente poder replicar os cenários na correção.

Exercício 1: Vamos extrair conhecimento de uma fábula

Neste exercício, deverás especificar uma ontologia OWL (podes usar o Protégé) para a história abaixo e os requisitos que lhe seguem. Lê o exercício todo antes de começares.

História

O Mestre Corvo, empoleirado no ramo de uma alta árvore, segurava no seu bico um apetitoso queijo. A Mestre Raposa, atraída pelo delicioso cheiro, aproximou-se da base da árvore e decidiu usar a sua astúcia.

"Bom dia, Senhor Corvo!", disse ela. "Que plumagem negra e brilhante o senhor tem! Se a sua voz for tão bela e encantadora quanto a sua aparência, o senhor é certamente o rei dos habitantes destes bosques."

O Corvo, sentindo-se imensamente vaidoso com os elogios, quis mostrar a sua magnífica voz. Ao abrir o bico de par em par para cantar, deixou cair a sua valiosa presa. A Raposa apanhou o queijo imediatamente e, com um sorriso trocista, disse:

"Meu bom senhor, aprenda que todo o adulator vive à custa de quem o escuta. Esta lição vale bem um queijo, sem dúvida!"

O Corvo, envergonhado e confuso, jurou para si mesmo, embora tarde demais, que nunca mais se deixaria enganar.

Ontologia (3 val.)

Especifica agora a ontologia que dê suporte ao conhecimento nesta história. Podes considerar conceitos como Personagem, Animal (e as suas subcategorias), Objeto, Sentimento ou Característica (como vaidade, astúcia, vergonha), e relações de posse, localização ou interação:

1. Define as Classes;
2. Define os atributos de cada Classe (data properties);
3. Define as relações que se podem estabelecer entre indivíduos da mesma classe ou de classes diferentes (object properties);
4. Cria os indivíduos com os seus atributos e as suas relações (ex: o indivíduo "Corvo", o indivíduo "Queijo", a relação do Corvo com a Árvore, etc.);
5. Deixa a ontologia num ficheiro TTL: `fabula.ttl`.

Queries (2 val. = 0.5 * 4)

Adicionalmente, especifica queries em SPARQL para responder às seguintes questões baseadas no conhecimento inferido da ontologia (coloca as queries, devidamente identificadas, num ficheiro de texto designado por `queries.txt`):

1. Quais são as personagens presentes na história e a que espécie animal pertencem?
2. Que objeto possuía o Corvo no início da história e onde ele estava localizado?
3. Quais são as características ou sentimentos associados à personagem "Raposa"?
4. Que personagem enganou o Corvo? (Recupera a relação de interação criada entre os dois indivíduos).

Exercício 2: Vamos construir uma ontologia a partir de alguns conjuntos de informação

Neste exercício, deverás especificar uma ontologia OWL a partir dum conjunto de datasets em JSON sobre o mundo dos **Jogos de Tabuleiro Modernos**.

Podes usar as ferramentas que entenderes para desenvolver e construir a ontologia: Protégé, scripts Python, ou outros.

Datasets sobre Jogos de Tabuleiro

O teu ponto de partida para a construção da ontologia serão os vários conjuntos de informação fornecidos em ficheiros JSON que se listam a seguir.

Jogos (Games)

O ficheiro `jogos.json` contém a informação base dos vários jogos de tabuleiro. Tem a seguinte estrutura:

```
[
  {
    "id": "catan",
    "name": "Catan",
    "category": "Family",
    "minPlayers": 3,
    "maxPlayers": 4,
    "playingTimeMinutes": 120,
    "descriptionEN": "Players take on the roles of settlers, each attempting to build and develop holdings while trading and acquiring resources."
  },
  ...
]
```

Autores (Designers)

O ficheiro `autores.json` contém a informação dos criadores (designers) dos jogos. Este ficheiro relaciona os autores com os jogos que conceberam. Tem a seguinte estrutura:

```
[
  {
    "id": "klaus-teuber",
    "name": "Klaus Teuber",
    "designedGames": [
      "catan",
      "catan-cities-and-knights"
    ]
  },
  ...
]
```

Editoras (Publishers)

O ficheiro `editoras.json` contém a informação das empresas responsáveis por editar e distribuir os jogos. Tem a seguinte estrutura:

```
[
  {
    "id": "kosmos",
    "name": "KOSMOS",
    "country": "Germany",
    "publishedGames": [
      "catan",
      "targi",
      "lost-cities"
    ]
  }
]
```

```
    },
    ...
]
```

Mecânicas (Mechanics)

O ficheiro `mecanicas.json` contem as mecânicas associadas à jogabilidade. Cada mecânica lista os jogos em que é aplicada. Tem a seguinte estrutura:

```
[
  {
    "id": "dice-rolling",
    "name": "Dice Rolling",
    "usedInGames": [
      "catan",
      "yahtzee",
      "king-of-tokyo",
      "stone-age"
    ]
  },
  ...
]
```

Prémios (Awards)

O ficheiro `premios.json` descreve grandes distinções recebidas pelos jogos. Tem a seguinte estrutura:

```
[
  {
    "id": "sdj-1995",
    "name": "Spiel des Jahres",
    "year": 1995,
    "wonByGame": "catan"
  },
  ...
]
```

Orientações/sugestões

Fica aqui uma sugestão para os conceitos iniciais e alguns requisitos para vos orientar na construção da ontologia OWL sobre jogos de tabuleiro:

Conceitos Iniciais

- Classes possíveis:

- Podem retirá-las dos ficheiros JSON, cada entidade representada num dos ficheiros poderá ser uma classe.
- Propriedades sugeridas (atributos e relações):
 - Podem retirá-las das chaves dos dicionários em cada um dos ficheiros JSON;
 - Cada par de um dos dicionários poderá ser um atributo da classe ou uma relação entre indivíduos de classes diferentes.

Atenção, isto são meras sugestões. São livres de estruturar a ontologia como entenderem. Podem dar o nome que entenderem às classes, atributos e propriedades.

Requisitos

- Identificador da ontologia (IRI):
 - A ontologia deverá ter o seguinte IRI: <http://www.di.uminho.pt/rpcw2026/Axxxxx> em que Axxxxx é o teu identificador de aluno.
- Estas são perguntas que a ontologia deve ser capaz de responder se bem modelada:
 - Quais são os jogos criados por um determinado autor?
 - Que mecânicas são utilizadas num determinado jogo?
 - Que jogos pertencem à categoria de "Strategy"?
 - Quais os jogos que já venceram o prémio "Spiel des Jahres"?
 - Quais as editoras sediadas na "Germany"?

Especificação e implementação

Vais agora, depois de analisares o material fornecido, as sugestões e os requisitos, especificar uma ontologia OWL.

De início, irás especificar a estrutura da tua ontologia: classes e propriedades. A seguir, irás povoar esta estrutura com a informação dos datasets em JSON disponibilizados.

Ontologia: estrutura base (2 val = 0.5 + 0.5 + 0.5 + 0.5)

Especifica agora a ontologia:

1. Define as Classes;
2. Define os atributos de cada Classe (data properties);
3. Define as relações que se podem estabelecer entre indivíduos da mesma classe ou de classes diferentes (object properties);
4. Grava o teu trabalho num ficheiro TTL: [boardgames_base.ttl](#).

Ontologia: povoamento/criação de indivíduos (3 val = 0.5 * 6)

Povoa agora a tua ontologia base com a informação presente nos datasets fornecidos. Nota: Podes usar scripts Python ou outra abordagem para agilizar o processo.

5. Jogos: [jogos.json](#);
6. Autores: [autores.json](#);

7. Editoras: `editoras.json`;
8. Mecânicas: `mecanicas.json`;
9. Prémios: `premios.json`;
10. Grava o teu trabalho num ficheiro TTL: `boardgames_ind.ttl` (cada ontologia que vais criando cresce incrementalmente em relação à anterior; esta deverá ter a estrutura base e os indivíduos).

Queries simples (3 val.: 0.5 * 6)

É importante que saibas que as queries que vais desenvolver a partir daqui serão testadas no GraphDB com a ontologia que criaste previamente e pela ordem em que aqui são pedidas.

11. Cria um repositório no teu GraphDB de nome "`boardgames`" e importa a ontologia criada;

Especifica as seguintes queries SPARQL no GraphDB e verifica os resultados (coloca estas queries num ficheiro "`sparql.txt`", cada query deverá estar completa tendo os prefixos necessários e deverá estar identificada com a alínea a que responde):

12. Quais são os jogos criados por um determinado autor (arbitra um nome, ex: "Klaus Teuber")?
13. Quais os prémios ganhos por um determinado jogo?
14. Que jogos pertencem à categoria Family?
15. Que jogos demoram mais de 60 minutos a jogar (`playingTimeMinutes`)?
16. Em que jogos entra a mecânica Dice Rolling?

Cálculo de distribuições (4 val. = 4 * 1)

Vais continuar a especificar queries, agora um pouco mais complexas:

17. Qual a distribuição de jogos por editora, ordenada decendentemente pela quantidade de jogos? (O resultado será uma lista de pares: Editora, número de jogos que publicou)
18. Qual a distribuição de jogos por categoria? (o resultado será uma lista de pares: Categoria, número de jogos)
19. Quais os jogos com mecânicas mais complexas? (o resultado será uma lista de jogos: id, nome e lista de mecânicas, apenas para jogos que têm mais de 3 mecânicas associadas)
20. Uma ludoteca escolar quer planear um evento e precisa da lista completa de autores e mecânicas envolvidas se utilizarem exclusivamente jogos da categoria Family. Produz uma lista sem repetições que inclua o nome do jogo, o nome dos seus criadores e as suas mecânicas.

Inferência de novo conhecimento (2 val. = 0.5 * 4)

Com a utilização de queries `CONSTRUCT` e queries `INSERT`, vais calcular novo conhecimento e nalguns casos acrescentar esse conhecimento à ontologia:

21. Cria uma nova relação na ontologia, `isPublishedBy`, que relaciona diretamente um jogo com a sua editora (o inverso da relação atual, caso tenhas seguido a orientação em `editoras.json`);
22. Cria uma query `CONSTRUCT` para calcular os novos triplos da nova relação `isPublishedBy`; Altera a query para uma query `INSERT` e acrescenta esses triplos à ontologia;

23. Cria uma nova relação na ontologia, `hasMechanic`, que relaciona um jogo com uma mecânica (o inverso da relação sugerida em `mecanicas.json`);
24. Cria uma query `CONSTRUCT` para calcular os novos triplos da nova relação `hasMechanic`; Altera a query para uma query `INSERT` e acrescenta esses triplos à ontologia.

Finalização (1 val.)

Para terminar:

25. Coloca na pasta do Git criada: a ontologia base, a ontologia povoada, o ficheiro com as queries e os scripts de migração (caso não tenhas usado scripts, explica como fizeste o povoamento no ficheiro `README.md`).

Bom trabalho e boa sorte!